

Experiment No: 01

Date of Performance: 16/01/26

Date of Submission: 06/02/26

Student Name: Aaryan Sharma

Roll No.: 16010123012 Batch: A1 Div:A

Staff In-charge:

TITLE: Encryption-Decryption using symmetric key

Objective:

To write a program for understanding Encryption-Decryption using classical cryptography (Playfair cipher/Substitution cipher Caesar cipher/affine cipher, Transposition cipher/rail fence cipher)

Expected OUTCOME of Experiment: Student will be able to

CO1: Explain various security goals, threats, vulnerabilities and controls

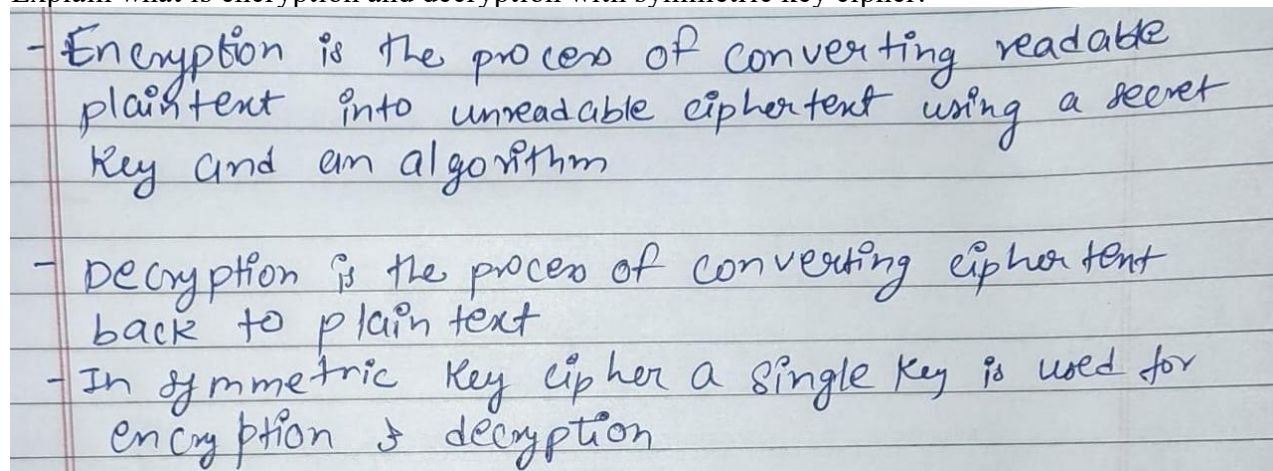
CO2: Apply various cryptographic algorithms for software security

Books/ Journals/ Websites referred:

1. Cryptography and Network Security, Behrouz.A. Fourouzan, Debdeep Mukhopadhyay, McGraw Hill Eighth Edition
2. Cryptography and Network Security: Principles and Practice William Stallings, Pearson Eighth Edition
3. Bernard Menezes Network Security and Cryptography, Cengage Learning Second Edition

Background Theory: (Answer in your handwriting)

Explain what is encryption and decryption with symmetric key cipher:



- Encryption is the process of converting readable plaintext into unreadable ciphertext using a secret key and an algorithm

- Decryption is the process of converting ciphertext back to plain text

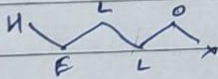
- In symmetric key cipher a single key is used for encryption & decryption

Explain Classical cryptography methods with example: Substitution / Transposition

ciphers:

- Substitution Cipher :- Replaces each character in the plaintext with another character according to a fixed system
eg. $A \rightarrow D, B \rightarrow E$ (key ± 3)
HELLO $\rightarrow$ KHOOR

- Transposition Cipher :- ~~Replaces~~ Rearranges the order of the character in the plaintext without changing identity
eg. HELLO $\rightarrow$ HLOELX



Playfair Cryptography with example:

- A digraph substitution cipher using 5x5 grid

Create Key Matrix
eg. MONARCHY

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	J
K	P	Q	S	T
U	V	W	X	Z

split plaintext into pairs
HELLO $\rightarrow$ HE LL OX

CF SU AV

Laboratory Work:

Program Implementation with algorithm:

1) Substitution Cipher

Caesar Cipher:

```
#include <iostream>
#include <string>
using namespace std;
```

```
int main() {
    string text;
    int key, choice;
```

```
    cout << "Caesar Cipher using ASCII\n";
```

```
cout << "1. Encrypt\n2. Decrypt\n";
cout << "Enter choice: ";
cin >> choice;

cin.ignore();
cout << "Enter text: ";
getline(cin, text);

cout << "Enter key: ";
cin >> key;

key = key % 26;

for (int i = 0; i < text.length(); i++) {
    char ch = text[i];

    if (ch >= 65 && ch <= 90) {
        if (choice == 1)
            ch = (ch + key > 90) ? ch + key - 26 : ch + key;
        else
            ch = (ch - key < 65) ? ch - key + 26 : ch - key;
    }

    else if (ch >= 97 && ch <= 122) {
        if (choice == 1)
            ch = (ch + key > 122) ? ch + key - 26 : ch + key;
        else
            ch = (ch - key < 97) ? ch - key + 26 : ch - key;
    }

    text[i] = ch;
}

cout << "Result: " << text << endl;

return 0;
}
```

Caesar Cipher using ASCII	Caesar Cipher using ASCII
1. Encrypt	1. Encrypt
2. Decrypt	2. Decrypt
Enter choice: 1	Enter choice: 2
Enter text: HelloWorld	Enter text: KoorZroug
Enter key: 3	Enter key: 3
Result: KoorZruog	Result: HelloWorld

Multiplicative Cipher:

```
#include <iostream>
#include <string>
using namespace std;

int modInverse(int key) {
    for (int i = 1; i < 26; i++) {
        if ((key * i) % 26 == 1)
            return i;
    }
    return -1;
}

int main() {
    string text, result = "";
    int key, choice;

    cout << "1. Encrypt\n2. Decrypt\nEnter choice: ";
    cin >> choice;
    cin.ignore();

    cout << "Enter text: ";
    getline(cin, text);

    cout << "Enter key: ";
    cin >> key;

    int invKey = modInverse(key);

    if (choice == 2 && invKey == -1) {
        cout << "Invalid key";
        return 0;
    }
}
```

```
for (char ch : text) {
    if (ch >= 65 && ch <= 90) {
        int x = ch - 65;
        if (choice == 1)
            result += char((x * key) % 26 + 65);
        else
            result += char((x * invKey) % 26 + 65);
    }
    else if (ch >= 97 && ch <= 122) {
        int x = ch - 97;
        if (choice == 1)
            result += char((x * key) % 26 + 97);
        else
            result += char((x * invKey) % 26 + 97);
    }
    else {
        result += ch;
    }
}

cout << "Result: " << result;
return 0;
}
```

1. Encrypt	1. Encrypt
2. Decrypt	2. Decrypt
Enter choice: 1	Enter choice: 2
Enter text: HelloWorld	Enter text: Vmhhq0qzhj
Enter key: 3	Enter key: 3
Result: Vmhhq0qzhj	Result: HelloWorld

Program Implementation with algorithm:

2) Transposition Cipher

Railfence Cipher:

```
def add_padding(message, rails):
    cycle = 2 * (rails - 1)
    if cycle == 0:
        return message
    pad_len = (cycle - (len(message) % cycle)) % cycle
    return message + ("X" * pad_len)
```

```
def rail_fence_encrypt(message, rails):
    message = message.replace(" ", "").upper()
    message = add_padding(message, rails)

    fence = [" " for _ in range(rails)]
    rail = 0
    direction = 1

    for char in message:
        fence[rail] += char
        rail += direction
        if rail == 0 or rail == rails - 1:
            direction *= -1

    return "".join(fence)

def rail_fence_decrypt(ciphertext, rails):
    ciphertext = ciphertext.replace(" ", "").upper()
    n = len(ciphertext)

    fence = [[" " for _ in range(n)] for _ in range(rails)]

    rail = 0
    direction = 1
    for i in range(n):
        fence[rail][i] = "*"
        rail += direction
        if rail == 0 or rail == rails - 1:
            direction *= -1

    index = 0
    for r in range(rails):
        for c in range(n):
            if fence[r][c] == "*":
                fence[r][c] = ciphertext[index]
                index += 1

    result = []
    rail = 0
    direction = 1
    for i in range(n):
        result.append(fence[rail][i])
        rail += direction
```

```
if rail == 0 or rail == rails - 1:  
    direction *= -1
```

```
return "".join(result).rstrip("X")
```

```
if __name__ == "__main__":  
    message = input("Enter message to encrypt: ")  
    rails = int(input("Enter number of rails: "))
```

```
    encrypted = rail_fence_encrypt(message, rails)  
    print("Encrypted:", encrypted)
```

```
    decrypted = rail_fence_decrypt(encrypted, rails)  
    print("Decrypted:", decrypted)
```

```
Enter message to encrypt: HelloWorld  
Enter number of rails: 3  
Encrypted: HOLELWRDXLOX  
Decrypted: HELLOWORLD  
  
=== Code Execution Successful ===
```

Single Columnar Cipher:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
vector<int> getOrder(string key) {  
    vector<pair<char,int>> v;  
    for (int i = 0; i < key.length(); i++)  
        v.push_back({key[i], i});
```

```
    sort(v.begin(), v.end());
```

```
    vector<int> order;  
    for (auto p : v)  
        order.push_back(p.second);
```

```
    return order;
```

```
}
```

```
string encrypt(string text, string key) {
```



```
int col = key.length();
int row = (text.length() + col - 1) / col;

vector<vector<char>> mat(row, vector<char>(col, 'X'));
int k = 0;

for (int i = 0; i < row; i++)
    for (int j = 0; j < col && k < text.length(); j++)
        mat[i][j] = text[k++];

vector<int> order = getOrder(key);
string cipher = "";

for (int idx : order)
    for (int i = 0; i < row; i++)
        cipher += mat[i][idx];

return cipher;
}

string decrypt(string cipher, string key) {
    int col = key.length();
    int row = cipher.length() / col;

    vector<vector<char>> mat(row, vector<char>(col));
    vector<int> order = getOrder(key);
    int k = 0;

    for (int idx : order)
        for (int i = 0; i < row; i++)
            mat[i][idx] = cipher[k++];

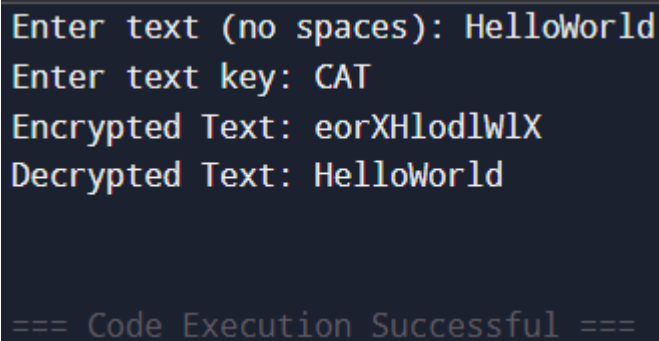
    string text = "";
    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++)
            text += mat[i][j];

    while (!text.empty() && text.back() == 'X')
        text.pop_back();
    return text;
}

int main() {
    string text, key;
```



```
cout << "Enter text (no spaces): ";  
cin >> text;  
  
cout << "Enter text key: ";  
cin >> key;  
  
string cipher = encrypt(text, key);  
cout << "Encrypted Text: " << cipher << endl;  
  
cout << "Decrypted Text: " << decrypt(cipher, key) << endl;  
  
return 0;  
}
```



```
Enter text (no spaces): HelloWorld  
Enter text key: CAT  
Encrypted Text: eorXHlodlWlX  
Decrypted Text: HelloWorld  
  
=== Code Execution Successful ===
```

Double Columnar Cipher:

```
#include <iostream>  
#include <vector>  
#include <algorithm>  
using namespace std;  
  
vector<int> getOrder(string key) {  
    vector<pair<char,int>> v;  
    for (int i = 0; i < key.length(); i++)  
        v.push_back({key[i], i});  
  
    sort(v.begin(), v.end());  
  
    vector<int> order;  
    for (auto p : v)  
        order.push_back(p.second);  
  
    return order;  
}  
  
string columnarEncrypt(string text, string key) {
```

```
int col = key.length();
int row = (text.length() + col - 1) / col;

vector<vector<char>> mat(row, vector<char>(col, 'X'));
int k = 0;

for (int i = 0; i < row; i++)
    for (int j = 0; j < col && k < text.length(); j++)
        mat[i][j] = text[k++];

vector<int> order = getOrder(key);
string cipher = "";

for (int idx : order)
    for (int i = 0; i < row; i++)
        cipher += mat[i][idx];

return cipher;
}

string columnarDecrypt(string cipher, string key) {
    int col = key.length();
    int row = cipher.length() / col;

    vector<vector<char>> mat(row, vector<char>(col));
    vector<int> order = getOrder(key);
    int k = 0;

    for (int idx : order)
        for (int i = 0; i < row; i++)
            mat[i][idx] = cipher[k++];

    string text = "";
    for (int i = 0; i < row; i++)
        for (int j = 0; j < col; j++)
            text += mat[i][j];

    return text;
}

int main() {
    string text, key1, key2;

    cout << "Enter plaintext (no spaces): ";
```

```
cin >> text;

cout << "Enter first key: ";
cin >> key1;

cout << "Enter second key: ";
cin >> key2;

string step1 = columnarEncrypt(text, key1);
string cipher = columnarEncrypt(step1, key2);

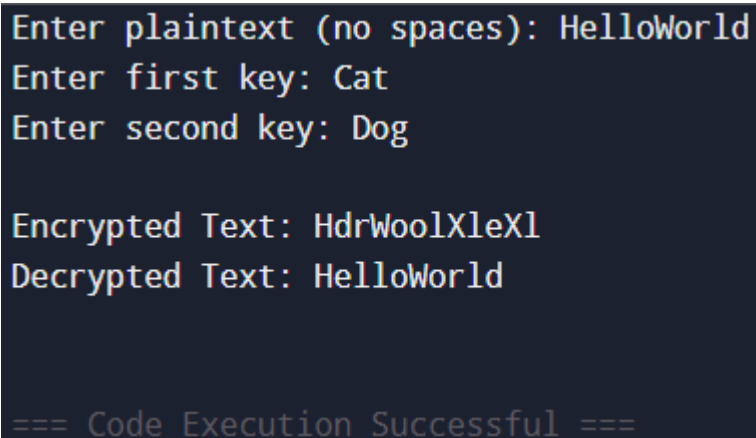
cout << "\nEncrypted Text: " << cipher << endl;

string dstep1 = columnarDecrypt(cipher, key2);
string decrypted = columnarDecrypt(dstep1, key1);

while (!decrypted.empty() && decrypted.back() == 'X')
    decrypted.pop_back();

cout << "Decrypted Text: " << decrypted << endl;

return 0;
}
```



```
Enter plaintext (no spaces): HelloWorld
Enter first key: Cat
Enter second key: Dog

Encrypted Text: HdrWoolXleXl
Decrypted Text: HelloWorld

=== Code Execution Successful ===
```

Conclusion:

In this experiment, classical symmetric key cryptographic techniques such as Caesar, Multiplicative, Rail Fence, Single Columnar, Double Columnar, and Hill cipher were successfully implemented. The encryption and decryption processes demonstrated how substitution and transposition methods secure data using a shared secret key. This practical work strengthened understanding of basic cryptographic principles, key management, and modular arithmetic, and achieved the objective of applying cryptographic algorithms for software security.

Post Lab Questions:

1. Encryption Decryptions (with screen shots of virtual lab)

Virtual Laboratory Experiments- (<http://cse29-iiith.vlabs.ac.in/>) :

(a) Breaking the Shift Cipher

PART I - Decryption Challenge

Decrypt the following ciphertext. You can use the tool beneath in PART III to simulate the Shift cipher.

Challenge 1 of 6
Decrypt: "haahjr ha khdu"

Ciphertext to be decrypted:

haahjr ha khdu

Next Ciphertext

PART II - Analysis Workspace

Do your rough work here:

2		tyyrlp	ty	lfbs	
3		exxego	ex	hear	Looks like English
4		dwdfn	dw	gdza	
5		cvcem	cv	fcyp	
6		buubdl	bu	ebxo	Looks like English
7		attack	at	dawn	★ Meaningful!
8		zsszlj	zs	czvm	
9		yrryai	yr	byul	
10		xqaxzh	xq	axtk	
11		amplia	am	plia	

Analysis Tools:

Frequency Analysis Test All Shifts Clear

 **PART III - Shift Cipher Simulator**

Plaintext:

attack at dawn

Shift: 7

 Encrypt ↓  Decrypt ↑

Ciphertext:

haahjr ha khdu

 **PART IV - Submit Your Solution**

Enter your solution:

attack at dawn

Key: 7

 Check my answer!

 **CORRECT!! Well done!**

You successfully decrypted: "haahjr ha khdu" → "attack at dawn" using shift 7.

 **VERIFICATION in Part III:**

-  Plaintext: "attack at dawn"
-  Shift: 7
-  Produces: "haahjr ha khdu"

(b) Breaking the Mono-alphabetic Substitution Cipher

Mono-Alphabetic Substitution Cipher Analysis

Decrypt the cipher text using frequency analysis and substitution techniques. Tools are provided to assist your cryptanalysis.

[Next Ciphertext](#) [Reset Workspace](#) [Calculate Frequencies](#)

Part 1: Ciphertext Analysis

Reference: English Letter Frequencies

Use this frequency table to guide your cryptanalysis:

a	b	c	d	e	f	g	h	i	j	k	l	m
8.167	1.49	2.702	4.253	12.702	2.228	2.015	6.094	6.966	0.153	0.772	4.035	2.406
n	o	p	q	r	s	t	u	v	w	x	y	z
6.749	7.507	1.929	0.095	5.987	6.327	9.056	2.750	0.978	2.369	0.150	1.974	0.074

Current Ciphertext:

The encrypted text to be analyzed

dkxyrb 1 = qagt vkr hxcov keur: suedr wn cehrg nrvvutp et vkr hvarhcto gvk kth nmerh, gkrt nkr tenedr x vuoutp, duvkg glavr hxcov gvk x yedory govdk hit yav. nkr leueng wv qagt x hxcov keur gert nqartob nkr luun x uetp gvb ve x dihvain kxuv gvk fatb uedorg qeehn el xuv namn. nkr luten x nfau orb ve x qeeh vee nfau leh kth ve luv, cuv ukheup glvdk nkr nrm xl zovhadvar pthart. nkr ukrt qndexro x cevur uccruug 'ghro fr', vkr detvryn el glvdk dxnr kth ve nihato vee nfau ve hndk vkr orb. x dser gvk 'rav fr' et wv dxnm kth ve phog ve nidk x vhrftqun nwr kth krag kmv vkr druvutp.

Ciphertext Frequency Analysis

Letter frequencies in the current ciphertext:

a	b	c	d	e	f	g	h	i	j	k	l	m
0.000	1.057	2.382	3.942	0.091	1.452	3.112	5.602	2.075	0.600	0.505	1.452	0.415
n	o	p	q	r	s	t	u	v	w	x	y	z
7.469	1.807	1.452	3.32	11.638	0.622	4.979	5.602	9.959	6.639	7.804	0.022	0.090

Part 2: Interactive Substitution Workspace

Substitution Scratchpad

Note: Cipher text is in lower case. When you replace characters, the final plaintext characters will be shown in upper case automatically.

Working Area:

Make substitutions and track your progress

CHAPTER 1 = DOWN THE RABBIT HOLE: ALICE IS BORED SITTING ON THE RIVERBANK WITH HER SISTER, WHEN SHE NOTICES A TALKING, CLOTHED WHITE RABBIT WITH A POCKET WATCH RUN PAST. SHE FOLLOWS IT DOWN A RABBIT HOLE AND SUDDENLY SHE FALLS A LONG WAY TO A STRANGE HALL WITH MANY LOCKED DOORS OF ALL SIZES. SHE FINDS A SMALL KEY ON A DOOR TOO SMALL FOR HER TO FIT, BUT THROUGH WHICH SHE SEES A BEAUTIFUL GARDEN. SHE THEN DISCOVERS A BOTTLE LABELLED 'DRINK ME', THE CONTENTS OF WHICH CAUSE HER TO SHRINK TOO SMALL TO REACH THE KEY. A CAKE WITH 'EAT ME' ON IT CAUSES HER TO GROW TO SUCH A TREMENDOUS SIZE HER HEAD HITS THE CEILING.

Substitution Tools

Replace cipher characters with plaintext:

This function is case insensitive and replaces cipher text (lower case) with plain text (upper case):

Replace cipher: with plaintext: [Apply](#)

Undo substitutions (case sensitive):

Use this function to undo unwanted changes by specifying exact characters:

Replace text: with text: [Replace](#)

Replacement History

You replaced r by E
You replaced v by T
You replaced k by H
You replaced x by A
You replaced d by C
You replaced y by P
You replaced e by C
You replaced y by P
You replaced n by K
You replaced c by B

Part 3: Solution Verification

 **Submit Your Solution**

After analyzing the ciphertext and making substitutions in the workspace above, enter your final solution below:

 Copy from Borakhatpad

 **Final Plaintext:**
Enter the complete decrypted message

chapter 1 - down the rabbit hole: alice is bored sitting on the riverbank with her sister, when she notices a talking, clothed white rabbit with a pocket watch run past. she follows it down a rabbit hole when suddenly she falls a long way to a curious hall with many locked doors of all sizes. she finds a small key to a door too small for her to fit, but through which she sees an attractive garden. she then discovers a bottle labelled 'drink me', the contents of which cause her to shrink too small to reach the key. a cake with 'eat me' on it causes her to grow to such a tremendous size

 **Solution Key:**
26 letters (a-z), each exactly once

xcdqrlpkwzouftayahnvigjbm

 Check Answer!

 **CORRECT!! Well done!**
 Plaintext: chapter 1 - down the rabbit hole: alice is bored sitting on the riverbank with her sister, when she notices a talking, clothed white rabbit with a pocket watch run past. she follows it down a rabbit hole when suddenly she falls a long way to a curious hall with many locked doors of all sizes. she finds a small key to a door too small for her to fit, but through which she sees an attractive garden. she then discovers a bottle labelled 'drink me', the contents of which cause her to shrink too small to reach the key. a cake with 'eat me' on it causes her to grow to such a tremendous size her head hits the ceiling.
 Key: xcdqrlpkwzouftayahnvigjbm

Part 4: Encryption Tool

 **Mono-Alphabetic Substitution Tool**

Use this tool to encrypt/decrypt with your own key and plaintext:

 **Plaintext:**
Your message to encrypt

defend the east wall of the castle

 **Substitution Key:**
26-letter substitution alphabet

xcdqrlpkwzouftayahnvigjbm

 Random Key

 Encrypt

 Decrypt

☐ Remove Punctuation

 **Ciphertext:**
Encrypted result

qrlrtq vkr rxnv gxuu el vkr dxnvur

2. Explain Hill cipher with an example and implement the program.

The Hill Cipher is a polygraphic substitution cipher based on linear algebra. Unlike simple substitution (which replaces 1 letter at a time), Hill cipher replaces blocks of letters (e.g., pairs or triplets) using matrix multiplication.

Key Concepts:

1. Key: A square matrix (e.g. times) of integers.

2. Block Size: Determined by the matrix size. If the key is times, we encrypt 2 letters at a time.
3. Formula: $C = (K \times P) \pmod{26}$
 - C: Ciphertext vector
 - K: Key Matrix
 - P: Plaintext vector

PROGRAM:

```
#include <iostream>
#include <vector>
using namespace std;

string hill_encrypt(string text, vector<vector<int>> &key, int n) {
    vector<int> msg(n), enc(n);
    for (int i = 0; i < n; i++) msg[i] = text[i] - 'A';

    for (int i = 0; i < n; i++) {
        enc[i] = 0;
        for (int j = 0; j < n; j++)
            enc[i] += key[i][j] * msg[j];
        enc[i] %= 26;
    }

    string cipher = "";
    for (int i = 0; i < n; i++)
        cipher += char(enc[i] + 'A');

    return cipher;
}

int main() {
    int n;
    cin >> n;

    vector<vector<int>> key(n, vector<int>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            cin >> key[i][j];

    string message;
    cin >> message;

    cout << hill_encrypt(message, key, n);
    return 0;
}
```

}

```
3
6 24 1
13 16 10
20 17 15
ACT
POH

...Program finished with exit code 0
Press ENTER to exit console.□
```